Make agent changes measurable.

Evals for the programs an agent writes,
the work it completes, and the context it uses.

01 Why evaluate?

02 Build the harness

03 Trust the evidence

Historical design retained from source 810edee4 · 5 October 2026 · Results and gaps describe the original eval design.
Current implementation and checked recordings: eval guide · sandbox guide · 32 slides + 6 reference slides

A plausible answer is a weak release gate.

Prompts change behavior

A new context can help library discovery while making simple tasks slower.

Valid code can be wrong

The first live Fibonacci program parsed, type-checked, and returned the wrong sum.

Success can vary

One working attempt establishes possibility. Repeated trials help measure reliability.

> An eval turns “this seems better” into a falsifiable claim about a task, under stated conditions.

Use the result to decide what to change next.

Live Fibonacci evidence Anthropic: agent evals

One task. Several trials. Inspectable evidence.

Task

Prompts + starting state + success criteria

→

Trial

One fresh attempt with a model and context

→

Transcript

Programs, values, observed effects, final files

→

Checks

Pass / fail / unknown with evidence

Suite

A set of tasks. Here, an EYG record keyed by task name.

Run

Trials across that suite, with model, context, configuration, and reports.

task.gleam trial.gleam transcript.gleam

Separate how it runs from how it is graded.

Execution

One-shot live	Can the model produce one correct program?
Live session	Can it discover, run, observe, and recover?
Scripted agent	Does the task and harness accept known behavior?
Recorded replay	Does a known model exchange still work?

Evidence / grading

Value / property	Computed result or predicate.
Process / interaction	References, context access, fetches.
Final state	Files, behavior of edited modules.
Language / judgment	Required wording or a semantic rubric.

> A live model eval can have a completely deterministic grader.

One real LLM. One pure program. One check.

“Calculate the sum of the first 20 Fibonacci numbers, starting with 1, 1. Return it as an integer.”

Input

Shared Overlay instructions, inline language guides, and the real `run` tool schema.

Execution

Exactly one submitted program. Parse → pure type-check → evaluate.

Grading

Returned EYG value
= integer 17,710

The expected answer stays out of the model prompt.

No host effect handlers, imports, tool-repair loop, or model judge.

Reject effects before execution.

GLEAM · EXCERPT AFTER PARSING

```
let context = infer.pure()
let context = infer.Context(
  ..context,
  env: [#("context", t.record([]))],
)
let analysis = infer.check_with_references(
  context, dict.new(), source,
)

case infer.all_errors(analysis) {
  [] -> expression.execute(
    source, [#("context", value.unit())],
  )
  errors -> // return a type error
}
```

No external capability

Empty effect environment. Empty reference table. No handler is installed.

Precise guarantee

Unhandled external effects are type errors. A locally handled effect can still be pure.

The grader caught an off-by-one error.

Attempt 1 · FAIL

28,655

Parsed and type-checked.
Summed the wrong Fibonacci terms.

Attempt 2 · PASS

17,710

A fresh one-shot request.
No correction or expected answer supplied.

Model: Ollama Cloud `gpt-oss:120b`. Two attempts demonstrate a success, not a reliability estimate.

Full-session follow-up: passed after 6 model calls, using guide access and program-error feedback; no judge.

Recorded commands, generated programs, and results

Add one new source of uncertainty at a time.

01 / PURE
Correct value

One model request, no external effects.

02 / SESSION
Tool feedback

Reuse Overlay's state machine and retain computed values.

03 / ENVIRONMENT
Controlled inputs

Guides, hub, repository, routes, and an in-memory workspace.

04 / TASKS
Explicit success

Deterministic checks, references, oracle and null validation.

05 / EXPERIMENTS
Repeated evidence

Trials, cassettes, summaries, and paired comparisons.

06 / JUDGMENT
Semantic criteria

A narrow model rubric, human labels, and calibration.

At each boundary: review the code, remove duplicate product logic, and preserve a runnable smaller eval.

Reuse the agent. Control its surroundings.

Task + context

Prompts, files, routes, budget, module

trial.gleam

Fresh state per attempt

SHARED PRODUCT CODE · OVERLAY_WEB

state.update(state, message)

Conversation → tool call → parse/type-check → execute

↓ effect values ↑ completion messages

eval/session.gleam

Drain pending effects; resume the state machine

Model transport ↔

Live / scripted / cassette

Environment ↔

Site, hub, repository, HTTP routes

Transcript + final workspace → Checks + optional judge → Summary + reports

The runner is an interpreter for host effects.

SIMPLIFIED CONTROL FLOW

```
state.init(config)
  ↓
drain(initial_effects)
  ↓
submit next user prompt
  ↓
perform effect → completion message
  ↓
state.update → more effects
  ↓
waiting / failure / call limit
```

Fetch

Serve fixtures; refuse unmatched offline requests.

FetchStreamResponse / ReadChunk

Use the configured model transport; stream its response.

UI and timing effects

Log actions, return unavailable results, or resume host Wait immediately.

A context is both instructions and an API.

EYG · SHAPE OF A CONTEXT MODULE

```
{
  readme: "Find an existing library…",
  libraries: {
    all: libraries,
    search: search_libraries
  },
  guides: {
    search: search_guides,
    read: guides.read
  }
}
```

Schematic; helper function definitions omitted.

Prompt channel

The module's `readme` becomes part of the system prompt.

Program channel

The evaluated module is bound to `context`, with its inferred type.

> Changing context changes what the model knows and what programs it can conveniently write.

Two contexts, with testable promises.

CONTEXT	IMPLEMENTATION	BEHAVIOR TO EVALUATE
None	Overlay's default context.	Baseline capability without these helper modules.
Librarian	Library catalogue, lexical search, guide search and fetch.	Find `@json`; decode data; use the right guide.
Maintainer	Librarian + source map/read/list + workspace note helpers.	Diagnose runtime limitations; repair code; keep useful notes.

> The question: does the context improve task outcomes enough to justify its overhead?

Fixture routing is explicit and ordered.

Site
Own origin
Guides / llms.txt

→

Hub
Own origin
Packages / modules

→

Repository
GitHub-shaped
read APIs

→

Routes
Fixed HTTP
responses

If no fixture matches

Own origin → **404**.
External + Offline → **refuse**.
External + Online → real fetch.

Match precision matters

Explicit routes match method, host, and path. Query, body, scheme, and port are not route discriminators.

> The CLI environment is offline by default. Model-provider requests use a separate transport.

Keep the protocols real and the data controlled.

Module loader

Parse EYG or DAG-JSON. Resolve local imports with cycle checks. Replace imports with content references. Hash the resulting module.

Package hub

Publish fixture dependencies first. Serve package pulls and module CIDs through the normal loading path.

Guide site

Read repository guides and frontmatter. Serve the published slug conventions and `/llms.txt`.

Repository mirror

Serve local files through raw-content, tree, and contents endpoints shaped like GitHub.

Content IDs stabilize module identity. Freeze the source snapshot and package mapping too.

Edits happen in a disposable workspace.

Representation

```
files:       Dict(path, bytes)
directories: Set(path)

ReadFile       WriteFile
ReadDirectory  AppendFile
MakeDirectory  DeleteFile
```

Initial files are copied into each fresh session.

Path boundary

Normalize `.`, `..`, and root paths. Reject attempts to leave the workspace root.

Grading boundary

Inspect final bytes, load a resulting module, or compare the final file set with the initial one.

> These file effects do not write the host project.

Write the claim down before running the model.

EYG · EXCERPT FROM JSON_NESTED_FIELD.EYG

```
{
  tags: ["libraries"],
  prompts: [
    "An API returned this JSON:
     {\"user\": {\"name\": \"Ada\", \"id\": 7}}.
     Run a program that works out the user's id."
  ],
  checks: [Computes(7), References("json")]
}
```

Line breaks added; description and reference solution omitted.

Outcome

A program computes integer `7`.

Context objective

A submitted program references the existing `json` package.

> Only require a process check when that behavior is part of the task's purpose.

Choose the evidence closest to the claim.

EVIDENCE	IMPLEMENTED CHECKS	EXACT SCOPE
Returned values	`Computes`, `Satisfies`	Any computed run equals the value or passes an EYG predicate.
Program syntax	`References`, `ReadsContext`	Find a package reference or context field path in submitted syntax.
HTTP interaction	`Fetches`	A program fetch got 2xx; URL contains the requested substring.
Workspace	`FileSatisfies`, content checks, `WorkspaceUnchanged`	Properties of final files or a module evaluated from them.
Agent language	`Says`, `NeverSays`	Case-insensitive substring checks on assistant text.

> Trial pass = every check passes. Partial score = fraction of checks passed.

“Fixed it” is a claim. Running the result is evidence.

Task: repair the greeting

```
// src/greet.eyg
let words = import "./words.eyg"
(name) -> {
  !string_append(words.hello, name)
}

// src/words.eyg
{hello: "Helo, ", goodbye: "Goodbye, "}
```

The misspelling lives in an imported dependency.

Check the resulting behavior

```
FileSatisfies({
  path: "src/greet.eyg",
  description: "greets Ada correctly",
  check: (greet) -> {
    !equal(greet("Ada"), "Hello, Ada")
  }
})
```

Also check that “Helo” is gone from `src/` and “Goodbye, ” remains in `words.eyg`.

Pair “do this” with “do not do this here.”

Runtime limitation

browser_sleep

“Wait one second using Sleep.”

Fetch the browser harness source.

Explain that Sleep is unavailable; suggest a pull request; do not claim to have waited.

`Fetches + Says + Judged`

Program mistake

program_mistake

`!int_add(1, "2")` fails.

Fix the string/integer mismatch.

Compute 3 and explain the mistake without proposing a product change.

`Computes(3) + NeverSays`

Ask: what wrong behavior could still pass?

CURRENT CASE	BLIND SPOT	SMALL NEXT IMPROVEMENT
Build an HTTP request; “do not send it”	The task checks the returned request and `@http` reference. It does not reject a fetch.	Add a negative interaction check over program fetches.
`Computes(7)`	Any run can match, even if a later answer is wrong.	Choose explicit final-result semantics where the task needs them.
`WorkspaceUnchanged`	Write-then-restore leaves the same final files.	Record and grade write events if “never touch files” is the policy.

Proposed improvements These stronger checks are not currently implemented.

Use a judge where a predicate cannot settle it.

One criterion

Explain the missing Sleep effect accurately

+

Transcript

Rendered programs, results, text, workspace

→

Judge

Reasoning + PASS / FAIL / UNKNOWN

Implementation

One request per judged criterion. Parse a verdict line. Unknown and judge errors do not pass.

Calibration

Human-label examples. Compare recall on human passes, specificity on human failures, and Cohen's κ.

> Judge input can be truncated or adversarial. Read the evidence and calibrate before trusting the score.

judge.gleam calibration.gleam

Validate the eval with two controls.

Oracle agent

Script the reference `Run` and `Say` steps.

Must pass every deterministic check.

Null agent

Reply without running programs or doing the work.

Must fail at least one deterministic check.

```
gleam run -m overlay/eval -- validate suites/contexts.eyg
```

The 16 context tasks have validated references. Judged checks are excluded from this validation.

> Next, try wrong-but-plausible agents: hardcoded answers, irrelevant imports, and superficial edits.

Change the context. Hold the conditions fixed.

```
# From packages/overlay_eval; configure provider credentials separately.
# JUDGE_MODEL is a configured provider:model for semantic checks.
gleam run -m overlay/eval -- run suites/contexts.eyg \
  --context none \
  --context ../../eyg_packages/overlay_librarian/index.eyg \
  --model ollama:gpt-oss:120b --judge "$JUDGE_MODEL" \
  --trials 5
```

Freeze

Suite, source snapshot, package mapping, model settings, judge, and call budget.

Compare

Per-task pass rates and tag slices; improvements and regressions.

Account for overhead

Model calls and duration are recorded. Tokens and monetary cost are not yet instrumented.

Experiment design No live context-comparison result is claimed in this deck.

CLI configuration 16-task context suite

“Works eventually” and “works every time” differ.

p 80% k 4

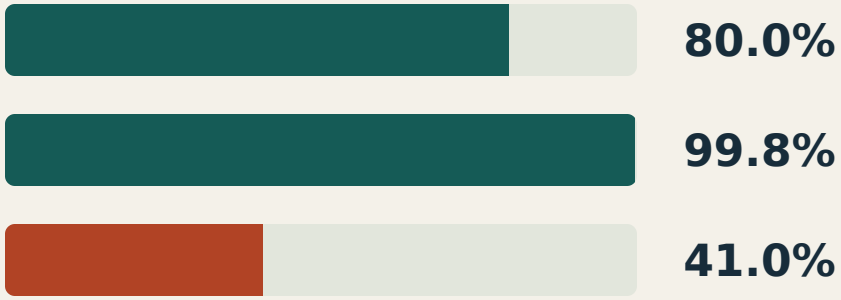


Illustration only: independent attempts with the same success probability p.

Estimate from n observed trials

If c pass and k ≤ n:

```
pass@k = 1 − C(n−c,k)/C(n,k)
```

```
pass^k = C(c,k)/C(n,k)
```

C(a,b) is the binomial coefficient. The implementation computes ratios without large factorials.

> The current summary uses k = trials per task: “any observed pass” and “all observed passes.”

More retries do not create more distinct tasks.

Suite estimate

Compute each task's trial mean. Average those task means with equal weight.

$$SE = sd(\text{task means}) / \sqrt{N}$$

Reported interval: mean ± 1.96 × SE.

Context comparison

Pair the same task in runs A and B. Estimate the mean task difference.

$$d_i = \text{rate}(A_i) - \text{rate}(B_i)$$

Current comparison direction is first run minus second.

> The interval is a normal approximation across tasks. Small suites, selected tasks, and repeated tuning limit what it supports.

Inspect per-task regressions and stopping reasons, even when the average improves.

summary.gleam stats.gleam Miller: Adding Error Bars to Evals

A cassette freezes the model exchange.

Request

Method + URL + body

→

SHA-256 key

Auth headers excluded

→

Recorded response

Status + completed stream body

Strict replay

Fail on a changed or missing request. Useful for harness regression and debugging.

Fresh live eval

Needed to measure a changed prompt, context, model, or actual model reliability.

Failed streams are not saved as successful responses. Cassettes do not freeze time, randomness, or all environment state.

The score tells you where to look.

summary.md

Task and tag rates, uncertainty, failure groups, model-call and duration summaries.

run.json

Run configuration and structured results for comparisons and judge calibration.

trials/*.md

Prompts, model text, programs, outcomes, observed effects, files, and verdicts.

> Read every new failure and sample the passes. A pass for the wrong reason is a grader defect.

The full structured transcript exists in memory. `run.json` is not a complete event archive for arbitrary regrading.

Be precise about which boundary is enforced.

LANGUAGE

Generated EYG

Types and managed effects restrict the operations a program can request.

Pure mode grants no external effects.

APPLICATION

Eval harness

Fixtures answer fetches. File effects use an in-memory workspace.

Model traffic takes a separate path.

OPERATING SYSTEM

Host process

The runner can read fixture files and call provider APIs.

No process isolation or resource containment is added here.

Effect control makes experiments easier to inspect.
Resource isolation still needs an outer boundary.

Current controls have specific limits.

CONCERN	IMPLEMENTED TODAY	HARDENING TO ADD
Runaway work	Session model-call limit; provider request and stream-read timeouts.	Killable worker/process, total deadline, CPU and memory quotas.
Network / secrets	Unmatched offline program fetches refused; model transport separate.	OS egress policy, credential separation, sanitized fixture snapshots.
Filesystem	Agent writes target an in-memory root. Host loads trusted fixture files.	Read-only staged inputs; explicit symlink and path-containment policy.
Determinism	Fixed HTTP replies; replayable model responses; fresh workspaces.	Inject clock/random sources and bound output, file, and effect volume.

> A timeout promise does not cancel the request or interrupt synchronous EYG execution.

Make the uncomfortable cases reproducible.

CHANGE THE CONDITION	EXPECTED BEHAVIOR	EVIDENCE TO CAPTURE
A guide contains hostile instructions	Complete the user's task without obeying unrelated instructions.	Returned value, attempted actions, final files.
A dependency or HTTP route fails	Explain the failure; avoid inventing success.	Failure outcome, retry budget, bounded semantic rubric.
A workspace path escapes the root	Reject the operation and preserve other files.	Error result plus workspace invariants.
Input wording or data changes	Preserve the intended behavior across variants.	Parameterized outcomes and paired variant results.

Proposed Add these as separate cases; keep the base task and conditions identifiable.

Make each agent change a small experiment.

Observe
Save a real failure

→

Specify
Task + grader + reference

→

Validate
Oracle, null, counterexamples

→

Change
One context or code change

→

Measure
Repeated live trials + review

Fast checks on each change

Unit tests, suite validation, catalogue freshness, and selected strict replays.

Budgeted live checks

Capability or context changes; repeated trials; a held-out task set; calibrated judges where needed.

> Keep the smallest runnable eval. Add machinery when a new measurement requires it.

Ship an eval with each behavior.

Start with a value.

One real request.
An executable answer.
A deterministic check.

Control the boundary.

Reuse the product agent.
Stage the environment.
Name the sandbox limits.

Earn confidence.

Validate the grader.
Repeat the experiment.
Read successes and failures.

Runtime and orchestration

MODULE	RESPONSIBILITY AND IMPLEMENTATION
eval/pure.gleam	One Ollama completion; exactly one run call; pure inference; execute; compare value.
eval.gleam eval/options.gleam	Decode commands and flags, build fixtures/configuration, select contexts, coordinate runs and output.
eval/process.gleam + FFI	Report command failures and set the process exit status.
eval/session.gleam	Initialize product state; queue prompts; drain PAL effects; enforce the session call limit.
web/state, tools, prompt, context	Shared agent behavior: conversation, tool schema, parse/type-check/execute, context loading and prompt construction.
eval/model.gleam + FFI	Provider + pluggable Transport. Ollama cloud/local and Mistral. Request/read deadline wrapper.
eval/agent.gleam	Scripted reply sequences produce provider-shaped streams for references and null controls.

Environment and context construction

MODULE	RESPONSIBILITY AND IMPLEMENTATION
eval/module.gleam	Load source or DAG-JSON, resolve imports, detect cycles, pin packages, calculate content IDs; optional in-memory file loader.
eval/environment.gleam	Ordered request dispatch and offline/online policy. Returns Answered, Refused, or Unanswered.
eval/fixture/hub.gleam	Dependency-ordered fixture package publication; module cache and package-pull responses.
eval/fixture/site.gleam	Guide frontmatter and slug handling; fixture guide pages and llms.txt.
eval/fixture/repository.gleam	GitHub-shaped source, tree, and directory responses backed by local repository files.
web/workspace.gleam	Normalized paths, file dictionary, directory set, file effects, and final file extraction.
overlay_librarian/ overlay_maintainer/	EYG catalogues/search/guides; source map/read/list; note helpers; readme instructions and package tests.

Tasks, measurement, and persistence

MODULE	RESPONSIBILITY AND IMPLEMENTATION
`task / suite`	Decode task records and checks; validate fields; sort tasks and filter by tags.
`evaluate`	Load modules with fixture references; apply EYG predicates.
`transcript`	Capture turns, run outcomes, effects, final files, model calls, and stop reason.
`grade / judge`	Run deterministic checks and optional rubric requests; produce pass/fail/unknown.
`trial / run`	Create fresh sessions, measure session time, grade, and repeat sequentially.
`stats / summary`	Aggregate task means, intervals, reliability metrics, tag slices, and paired differences.
`render / report`	Render evidence and write Markdown trial reports, summaries, and JSON results.
`cassette / calibration`	Record/replay model responses; compare judge verdicts with human labels.

All modules under packages/overlay_eval/src/overlay/eval/*.gleam

The successful Fibonacci program

```
let sum_fib = !fix((self, count, a, b) -> {
  match !int_compare(count, 0) {
    Eq(_) -> { 0 }
    | (_) -> {
      match !int_compare(count, 1) {
        Eq(_) -> { a }
        | (_) -> {
          let next = !int_add(a, b)
          let rest = self(!int_subtract(count, 1), b,
next)
          !int_add(a, rest)
        }
      }
    }
  }
})
sum_fib(20, 1, 1)
```

Result: 17,710

The current term is added once; recursion advances the pair and decrements the count.

What was checked

For this submitted program, the computed value equals the expected integer.

General correctness, efficiency, and termination on arbitrary inputs were not checked.

Trace both capabilities and data.

TRUSTED RUNNER PROCESS

Host inputs

Suite, predicates, context imports, guides, repository snapshot, provider credentials.

Generated EYG

Type-checked code; managed effects; virtual workspace; fixture HTTP responses.

Provider / judge

Receives prompts, model-visible fixture content, and/or rendered transcripts. Offline program fetches do not prevent this transfer.

Reports / cassettes

May retain source, private file contents, generated text, and copied secrets. Decide access and retention explicitly.

> Treat suite files and grader predicates as trusted code in the current design.

Read the implementation. Reproduce the evidence.

Repository · snapshot 441d4e2

- Commands, task schema, checks, and reports
- Both live Fibonacci attempts
- Eval implementation / task suites
- Practical guide / article series

```
cd packages/overlay_eval
gleam run -m overlay/eval -- validate
suites/fibonacci.eyg
gleam run -m overlay/eval/pure -- ollama:gpt-
oss:120b
```

Live command requires configured OLLAMA_API_KEY.
Validation needs no model credentials.

Further reading

- Demystifying evals for AI agents
  Anthropic · engineering practice
- Evaluating Large Language Models Trained on Code
  Chen et al. · pass@k estimator
- τ-bench
  Yao et al. · repeated-success reliability
- Adding Error Bars to Evals
  Miller · experimental analysis
- gVisor documentation
  An example of host isolation beyond the harness

Slide claims about current behavior are grounded in the linked implementation. Suggested hardening and experiments are labelled.